

Лабораторная работа №2 ВЗАИМОДЕЙСТВИЕ ПРОЦЕССОВ

Цель работы: научиться осуществлять передачу произвольных данных между параллельно выполняющимися процессами.

Взаимодействие нескольких параллельных процессов часто вызывает необходимость обмена данными между ними. При этом возникает следующая проблема: каждый процесс имеет собственное адресное пространство, при этом переменные одного процесса недоступны другим процессам. Обычно для передачи данных между процессами используют глобальные объекты ядра, выделение общих сегментов памяти, сокеты, каналы и т.д.

Среди механизмов передачи данных между процессами в Windows наиболее часто используются каналы и разделяемая память.

Как и в Linux, каналы в Windows также применяются для связи двух процессов. Процесс, создающий канал, называют *сервером*, а процесс, использующий этот канал, – *клиентом*. Каналы в Windows бывают двух типов: анонимные и именованные. Анонимные каналы быстры и универсальны, однако процессу-клиенту сложнее получить дескриптор такого канала, кроме того, они не поддерживают дуплексную передачу данных и не работают в сетях. Именованные каналы не имеют подобных недостатков, однако немного более тяжеловесны для операционной системы.

Использование каналов в Windows во многом подобно таковому в Linux. Рассмотрим основные функции для работы с каналами. Создание анонимного канала:

BOOL CreatePipe(PHANDLE hReadPipe, PHANDLE hWritePipe, LPSECURITY_ATTRIBUTES lpPipeAttributes, DWORD nSize);

Функция создает анонимный канал, используя следующие параметры:

- *hReadPipe* – дескриптор чтения;
- *hWritePipe* – дескриптор записи;
- *lpPipeAttributes* – атрибуты защиты;
- *nSize* – количество байт, резервируемых для канала.

Создание именованного канала:

HANDLE CreateNamedPipe(LPCTSTR lpName, DWORD dwOpenMode, DWORD dwPipeMode, DWORD nMaxInstances, DWORD nOutBufferSize, DWORD nInBufferSize, DWORD nDefaultTimeout, LPSECURITY_ATTRIBUTES lpSecurityAttributes);

Функция принимает следующие параметры:

- *lpName* – имя канала. Как правило, представляет собой строку вида: «\\Имя сервера\pipe\Имя канала». При использовании именованных каналов на локальной машине данная строка сокращается до «\\.\pipe\Имя канала»;
- *dwOpenMode* – режим открытия канала;
- *nMaxInstances* – максимальное количество реализаций канала;
- *nOutBufferSize* – размер выходного буфера;
- *nInBufferSize* – размер входного буфера;
- *nDefaultTimeOut* – время ожидания, в миллисекундах;
- *lpSecurityAttributes* – атрибуты защиты.

С целью дальнейшего использования канала с серверной и клиентской сторон применяют следующие функции: *ConnectNamedPipe()* (для именованных каналов), *CreateFile()*, *ReadFile()*, *WriteFile()*. После установления соединения через канал с ним можно работать как с обычным файлом.

Разделяемая память

Еще один способ передать данные между процессами – использование файлов, отображаемых в память. При этом один процесс создает специальный объект, «файловую проекцию» (File Mapping), выделяя область памяти, которая связывается с определенным файлом и в дальнейшем может быть доступна глобально из других процессов.

Для передачи данных между процессами такое решение может показаться избыточным. Однако нет необходимости в специальном файле, проекция может использоваться исключительно для выделения виртуальной памяти без привязки к конкретному файлу.

Последовательность действий при этом такова. Для создания файловой проекции используется следующая функция:

HANDLE WINAPI CreateFileMapping(HANDLE hFile, LPSECURITY_ATTRIBUTES lpAttributes, DWORD flProtect, DWORD dwMaximumSizeHigh, DWORD dwMaximumSizeLow, LPCTSTR lpName);

Данная функция принимает параметры:

- *hFile* – дескриптор файла, связываемого с выделяемой областью памяти. Если связывать файл не нужно, а функция используется только для выделения общей области памяти, данный параметр устанавливается в *INVALID_HANDLE_VALUE*. При этом в качестве проецируемого файла будет использоваться системный файл подкачки. В этом случае также необходимо явно указать размер выделяемой памяти в параметрах *dwMaximumSizeHigh* и *dwMaximumSizeLow*;
- *lpAttributes* – атрибуты безопасности;
- *flProtect* – флаги защиты выделенной области;

- *dwMaximumSizeHigh* – старшее слово размера выделяемой памяти, в байтах;
- *dwMaximumSizeLow* – младшее слово размера выделяемой памяти, в байтах;
- *lpName* – имя объекта «файловая проекция».

После создания файлового отображения необходимо получить его адрес в памяти для того, чтобы записать туда передаваемые данные. Получить адрес можно при помощи следующей функции:

LPVOID MapViewOfFile(HANDLE hFileMappingObject, DWORD dwDesiredAccess, DWORD dwFileOffsetHigh, DWORD dwFileOffsetLow, SIZE_T dwNumberOfBytesToMap);

Принимаемые параметры:

- *hFileMappingObject* – дескриптор файловой проекции, полученный от предыдущей функции;
- *dwDesiredAccess* – режим доступа к области памяти;
- *dwFileOffsetHigh* – старшее слово смещения файла, с которого начинается отображение;
- *dwFileOffsetLow* – младшее слово смещения;
- *dwNumberOfBytesToMap* – число отображаемых байт. Если параметр равен нулю, отображается весь файл.

Данная функция возвращает указатель на спроецированную область памяти. После его получения можно записывать в полученную общую память необходимые данные.

Процесс-клиент должен получить доступ к памяти, выделенной другим процессом. Здесь необходимо учитывать, что файловая проекция уникально идентифицируется именем, указанным в функции *CreateFileMapping()*. Для получения дескриптора проекции следует воспользоваться функцией *OpenFileMapping()*. Далее по полученному дескриптору при помощи функции *MapViewOfFile()* можно получить указатель на искомую область памяти.

ЗАДАНИЕ

Необходимо создать два процесса: клиентский и серверный. Серверный процесс ждет ввода пользователем текстовой строки и по нажатии клавиши Enter инициируются следующие действия:

- клиентский процесс ожидает уведомления о том, что серверный процесс готов начать передачу данных (синхронизация);
- серверный процесс передает полученную от пользователя строку клиентскому процессу, используя либо каналы, либо сегменты разделяемой памяти / файловые проекции;
- клиентский процесс принимает строку и выводит ее на экран;

- серверный процесс ожидает уведомления от клиентского процесса об успешном получении строки;

- серверный процесс вновь ожидает ввода строки пользователем и т.д.

В данной работе продолжается освоение синхронизации процессов. Уведомление процессов должно производиться посредством семафоров. Реализация механизма непосредственной передачи данных остается на выбор студента, однако в теории освоены должны быть все варианты.